



Lab 05: Build Infrastructure with Terraform - Challenge Lab

Challenge Lab | Build Infrastructure with Terraform | GSP345



This is a Challenge Lab. No step-by-step instructions provided.

Skills from Labs 01-04 were applied independently to complete all tasks.

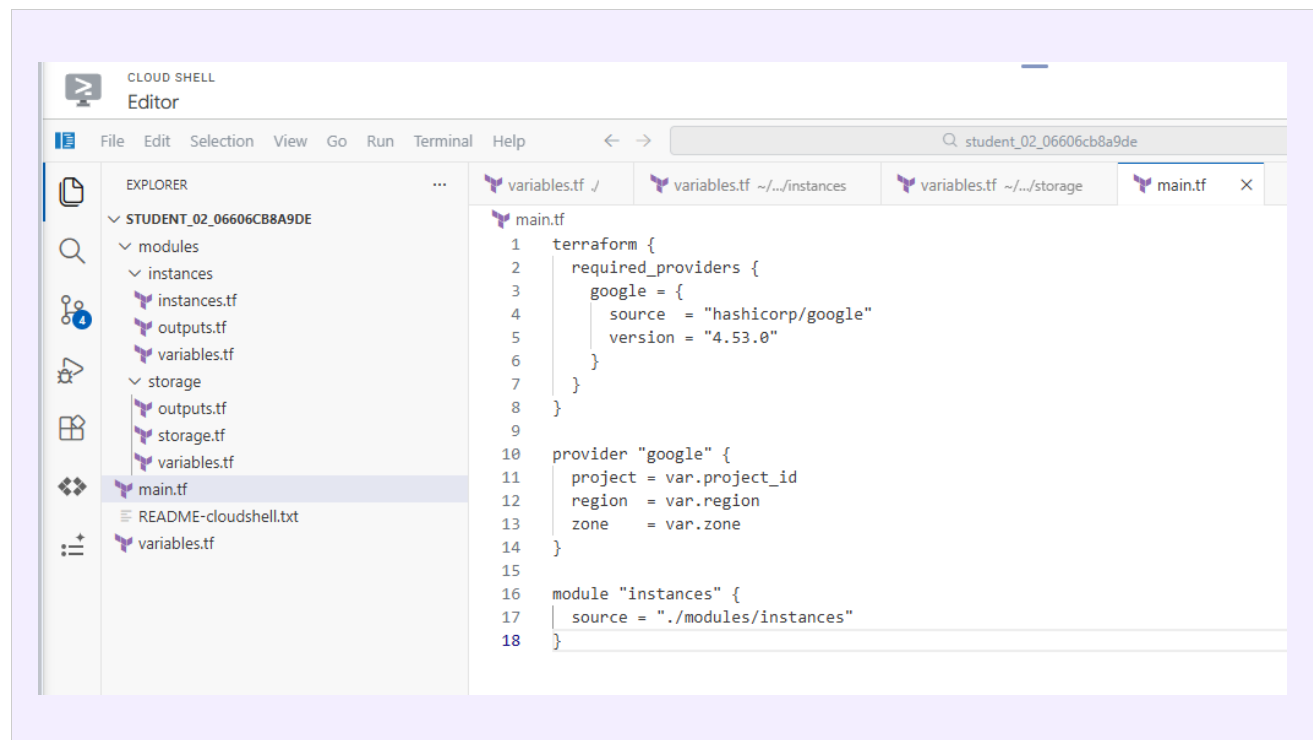
Overview

Imported existing VM instances into Terraform, built custom modules, configured a Cloud Storage remote backend, used a Registry module to create a VPC with subnets, and added a firewall rule to allow connections between instances.

Task 1 - Create Configuration Files

Created the following folder and file structure for the Terraform project:

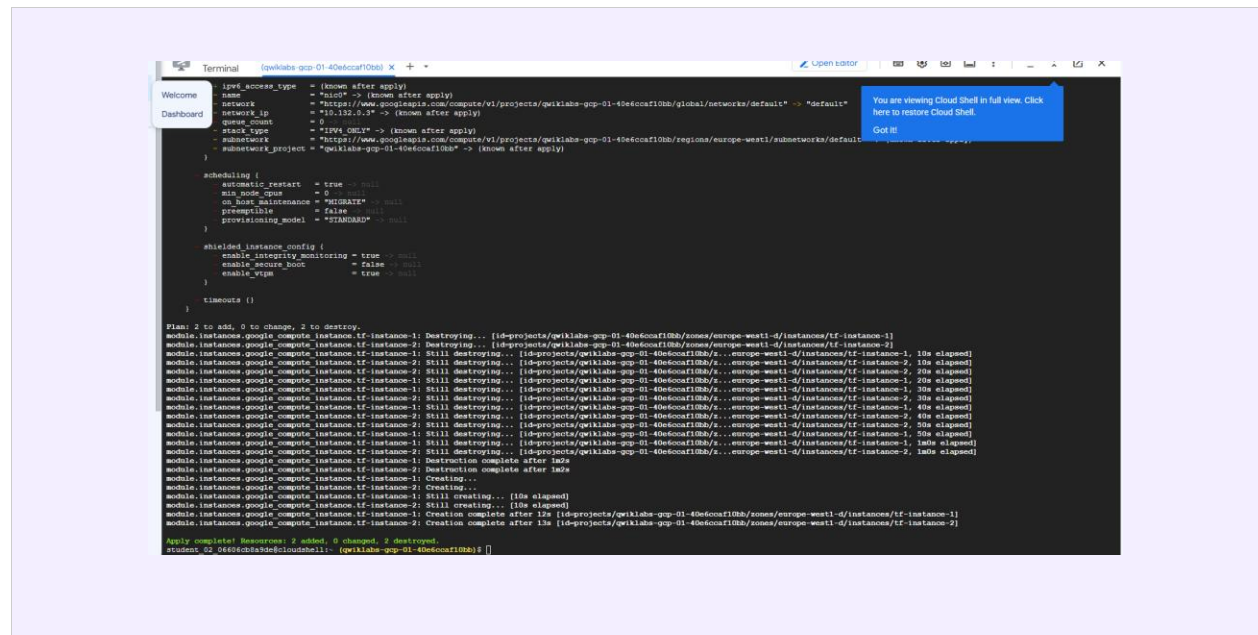
```
main.tf
variables.tf
modules/instances/instances.tf
modules/instances/outputs.tf
modules/instances/variables.tf
modules/storage/storage.tf
modules/storage/outputs.tf
modules/storage/variables.tf
```



Task 2 - Import Infrastructure

Imported two pre-existing VM instances tf-instance-1 and tf-instance-2 into the instances module.

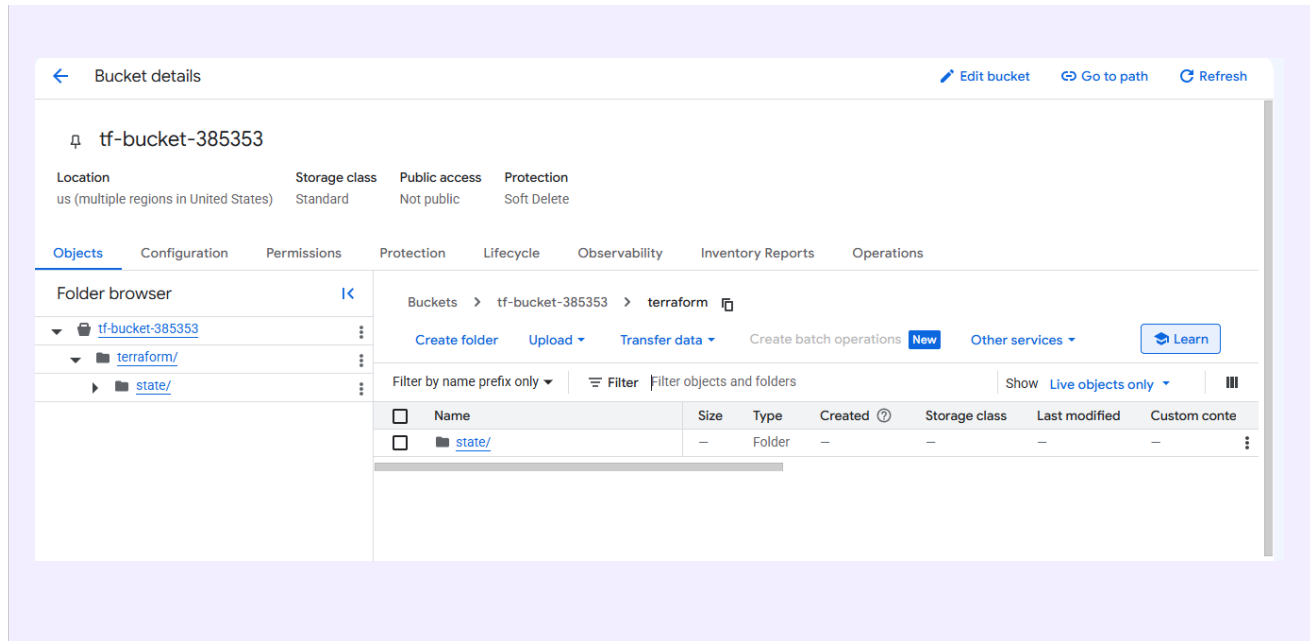
```
terraform import module.instances.google_compute_instance.tf-instance-1
[INSTANCE_ID_1]
terraform import module.instances.google_compute_instance.tf-instance-2
[INSTANCE_ID_2]
terraform apply
```



Task 3 - Configure Remote Backend

Created a Cloud Storage bucket in the storage module and configured it as the remote backend.

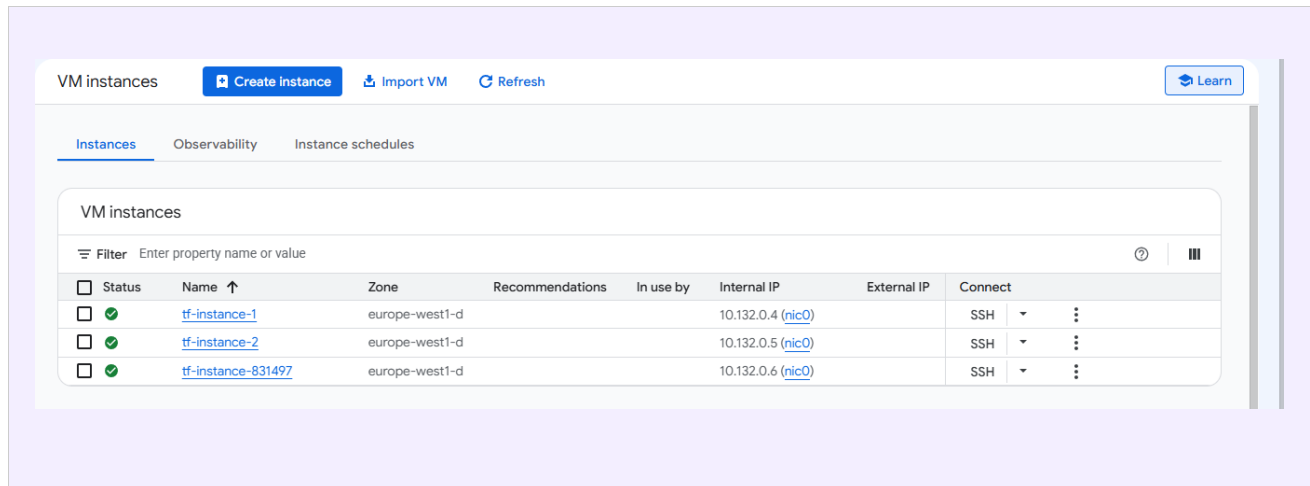
```
terraform {
  backend "gcs" {
    bucket = "[BUCKET_NAME]"
    prefix = "terraform/state"
  }
}
terraform init -migrate-state
```



Task 4 - Modify and Update Infrastructure

Updated both instances to e2-standard-2 and added a third instance.

```
machine_type = "e2-standard-2"
terraform apply
```



Task 5 - Destroy Resources

Removed the third instance from configuration and applied to destroy it.

```
terraform apply
```

Task 6 - Use Module from Registry

Added the terraform-google-modules/network module to create a VPC with two subnets, then connected each instance to its subnet.

```

module "vpc" {
  source      = "terraform-google-modules/network/google"
  version     = "10.0.0"
  project_id  = var.project_id
  network_name = "[VPC_NAME]"
  routing_mode = "GLOBAL"
  subnets = [
    { subnet_name = "subnet-01" subnet_ip = "10.10.10.0/24" subnet_region = "[REGION]" },
    { subnet_name = "subnet-02" subnet_ip = "10.10.20.0/24" subnet_region = "[REGION]" }
  ]
}

```

SMTP port 25 disallowed in this project. [Learn more](#)

VPC networks Manage flow logs

Filter Enter property name or value

☐	Name ↑	Subnets	MTU ?	Mode	IPv6 ULA range	Gateways	Flow log configs ?	Firewall rules	Global dynamic routing
☐	default	24	1460	Auto				4	Off
☐	tf-vpc-189290	2	1460	Custom				0	On

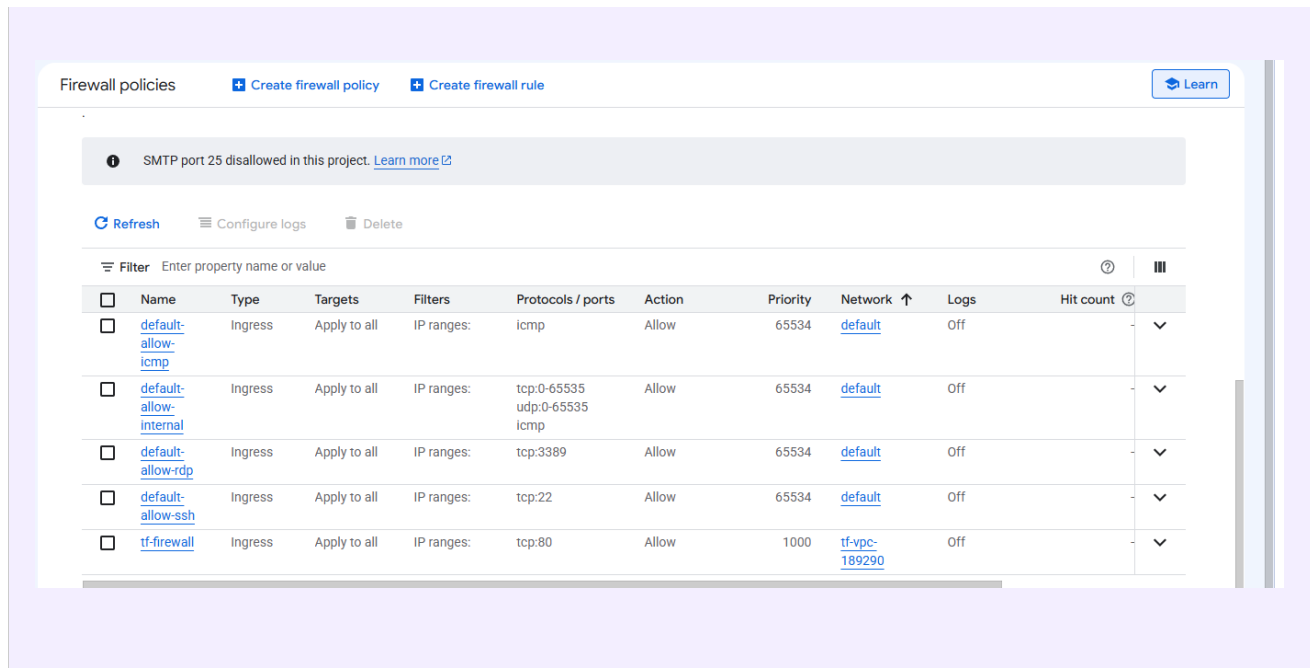
Task 7 - Configure Firewall

Created a firewall rule to allow TCP port 80 ingress from all IP ranges on the VPC network.

```

resource "google_compute_firewall" "tf-firewall" {
  name     = "tf-firewall"
  network = "projects/[PROJECT_ID]/global/networks/[VPC_NAME]"
  allow {
    protocol = "tcp"
    ports    = ["80"]
  }
  source_ranges = ["0.0.0.0/0"]
}

```



What I Learned

- Terraform modules allow you to organize and reuse infrastructure configuration
- Importing existing resources is a multi-step process requiring manual config writing
- Remote backends enable team collaboration and prevent state corruption
- Registry modules save time and enforce best practices for common infrastructure
- Resource dependencies are handled automatically when using module outputs
- Firewall rules can reference network self_link or ID from state

Resources

Terraform Docs: <https://developer.hashicorp.com/terraform/docs> | Lab ID: GSP345